



ipb

INSTITUTO POLITÉCNICO DE BRAGANÇA
Escola Superior de Tecnologia e Gestão

Engenharia Informática - Arquitetura de Computadores
Teste Intermédio de 05/05/2017 - [1h30m]

1. (Cap 4) Considere o seguinte programa, codificado em assembly MARIE:

```
//Etiqueta    // Diretivas, Instruções e Dados
              ORG 100
              Load X
              Store Temp
              JnS Subr
              Store X
              Load Y
              Store Temp
              JnS Subr
              Store Y
              Halt
X,            Dec 20
Y,            Dec 48
Temp,        Dec 0
Subr,        Hex 0
              Clear
              Load Temp
              Add Temp
              JumpI Subr
```

Códigos de operação (HEX) e mnemónicas do assembly MARIE

(1) Load X	(2) Store X	(3) Add X	(4) Subt X
(5) Input	(6) Output	(7) Halt	(9) Jump X
(0) JnS X	(A) Clear	(B) AddI X	(C) JumpI X
(8) Skipcond X , em que: (X == 000) => (AC<0 ?) (X == 400) => (AC==0?) (X == 800) => (AC>0 ?)			

Descrições RTL das instruções Load X, Store X e JnS X:

Load X	MAR ← X ; MBR ← Memory[MAR] ; AC ← MBR
Store X	MAR ← X ; MBR ← AC ; Memory[MAR] ← MBR
JnS X	MBR ← PC ; MAR ← X ; Memory[MAR] ← MBR; MBR ← X ; AC ← 1 ; AC ← AC+MBR; PC ← AC

Assemble o programa anterior e apresente na sua folha de resolução:

- a) a tabela de símbolos que resulta da primeira passagem;
- b) o resultado final da assemblagem, em formato hexadecimal, que resulta da segunda passagem;
- c) os valores dos registos PC, IR, MAR, MBR e AC após a execução **c1)** da primeira instrução (Load X), **c2)** da segunda instrução (Store Temp), e **c3)** da terceira instrução (JnS Subr).

2. (Cap 4)

Considere o seguinte código C de um programa que lê dois números inteiros (aa e bb) e determina o resto (rr) da divisão (inteira) do primeiro pelo segundo, recorrendo a uma função (Resto).

```
#include <stdio.h>

int Resto (int a, int b) {
    int r;

    r = a;
    while ( r - b + 1 > 0)
        r = r - b;
    return(r);
}

int main() {
    int aa, bb, rr;

    scanf("%d", &aa);
    scanf("%d", &bb);
    rr = Resto(aa, bb);
    printf("%d\n", rr);
}
```

Traduza, para assembly MARIE, o código C: **a)** da função main, **b)** da função Resto.

3. (Cap 4)

Considere o seguinte código C de um programa que verifica, para cada elemento de um array (**aa**), se o mesmo é num número ímpar ou par, apresentando no ecrã 1 ou 0, respetivamente. O programa em causa assume a disponibilidade da função **Resto**, previamente introduzida na questão 2.

```
int Impar (int n) {
    int p;

    p = Resto(n, 2);
    return(p);
}

int main() {
    int aa[5]={1,2,3,4,5}, ii;

    for (ii=0; ii<5; ii++)
        printf("%d\n", Impar(aa[ii]));
}
```

O código abaixo é uma implementação em MARIE incompleta da função **main** anterior. Apresente, na sua folha de resolução, os **BLOCOS** de código MARIE em falta, de forma a respeitar o código C anterior.

Loop,	ORG 100	
	Load	ArrayCellIndex
	Subt	Five
	Skipcond	000
	Jump	EndLoop
	/ BLOCO 1 : coloca em ArrayCellAddress o endereço de aa[ii]	

	/ BLOCO 2 : coloca em AC o conteúdo de aa[ii]	

	Store	Impar_n
	JnS	Impar
	Load	Impar_p
	Output	
	/ BLOCO 3 : incrementa o índice de acesso ao array (ii++)	

	Jump	Loop
EndLoop,	Halt	
	Dec	1 / aa[0]
	Dec	2 / aa[1]
	Dec	3 / aa[2]
	Dec	4 / aa[3]
	Dec	5 / aa[4]
ArrayBaseAddress,	Hex	_____ / BLOCO 4 : endereço de aa[0]
ArrayCellAddress,	Hex	000 / endereço de aa[ii]
ArrayCellIndex,	Dec	0 / índice ii
One,	Dec	1 / constante 1
Five,	Dec	5 / constante 5